Final Report for Load Forecasting Competition

https://github.com/ayenyein-thu/LopezThu_ENV797_TSA_ForecastCompetition_S25/

Aye Nyein Thu, Alex Lopez

Contents

1 Importing Data

```
Demand <- read_excel(path = "./Data/Raw/load.xlsx", sheet = "Sheet1",
                     col_names = TRUE)

Humidity <- read_excel(path = "./Data/Raw/relative_humidity.xlsx", sheet = "Sheet1",
                       col_names = TRUE)

Temperature <- read_excel(path = "./Data/Raw/temperature.xlsx", sheet = "Sheet1",
                          col_names = TRUE)
```

2 Data Wrangling

2.1 Demand

```
# Format the hourly data set
Hourly_Demand <- Demand %>%
  pivot_longer(cols = starts_with("h"), names_to = "hour", values_to = "Hourly_demand") %>%
  mutate(Hour = as.numeric(sub("h", "", hour))) %>%
  mutate(Date = ymd(date)) %>%
  select(Date, Hour, Hourly_demand)

#summary(Hourly_Demand) # 6 NA's

# Format the daily data set
Daily_Demand  <- Hourly_Demand %>%
  filter(!is.na(Hourly_demand)) %>%
  group_by(Date) %>%
  summarise(Daily_demand = mean(Hourly_demand))
```

2.2 Humidity

```
# Format the hourly data set
Hourly_Humidity <- Humidity %>%
  mutate(Date = ymd(date), Hour = hr,
         Hourly_humidity = rowMeans(select(.,rh_ws1:rh_ws28), na.rm = TRUE)) %>%
  select(Date, Hour, Hourly_humidity)

# Format the daily data set
Daily_Humidity  <- Hourly_Humidity %>%
  group_by(Date) %>%
  summarise(Daily_humidity = mean(Hourly_humidity))
```

2.3 Temperature

```r
# Format the hourly data set
Hourly_Temperature <- Temperature %>%
  mutate(Date = ymd(date), Hour = hr,
         Hourly_temp = rowMeans(select(.,t_ws1:t_ws28), na.rm = TRUE)) %>%
  select(Date, Hour, Hourly_temp)

# Format the daily data set
Daily_Temperature  <- Hourly_Temperature %>%
  group_by(Date) %>%
  summarise(Daily_temp = mean(Hourly_temp))
```

2.4 Combining Daily and Hourly Data Sets

```r
# Combine the hourly data set
Hourly_Electricity <- Hourly_Demand %>%
  left_join(Hourly_Humidity, by = c("Date", "Hour")) %>%
  left_join(Hourly_Temperature, by = c("Date", "Hour"))

#summary(Hourly_Electricity) # 6 NA's

# Combine the daily data set using the simple average
Daily_Electricity <- Daily_Demand %>%
  left_join(Daily_Humidity, by = "Date")  %>%
  left_join(Daily_Temperature, by = "Date")
```

2.5 Creating Multi-Seasonal Time Series Objects

```r
ts_Hourly_Electricity <- msts(Hourly_Electricity[,3:5],
                              seasonal.periods = c(24,168,8766),
                              start = c(2005, 1, 1, 1)) # Added the hour

ts_Daily_Electricity <- msts(Daily_Electricity[,2:4],
                              seasonal.periods = c(7, 365.25),
                              start = c(2005, 1, 1))
```

2.6 Handling NA's and Outliers

```r
# Check missing values
#sum(is.na(ts_Hourly_Electricity))
#sum(is.na(ts_Daily_Electricity))

# Check outlier by test
#outlier(ts_Hourly_Electricity)
#grubbs.test(ts_Hourly_Electricity[,1]) # outlier
#grubbs.test(ts_Hourly_Electricity[,2]) # not outlier
#grubbs.test(ts_Hourly_Electricity[,3]) # not outlier
```

```
#outlier(ts_Daily_Electricity)
#grubbs.test(ts_Daily_Electricity[,1]) # not outlier
#grubbs.test(ts_Daily_Electricity[,2]) # not outlier
#grubbs.test(ts_Daily_Electricity[,3]) # not outlier

# Handle NA's and outliers for just the hourly data
ts_Hourly_Demand_clean <- tsclean(ts_Hourly_Electricity[,1])
#outlier(ts_Hourly_Demand_clean)
#grubbs.test(ts_Hourly_Demand_clean) # not outlier
```

2.7 Setting Training and Testing Windows

```
# DAILY
n_for = 59 #days for January and February

# Create time series for data up until February 28, 2010
ts_Daily_Electricity_Feb <- subset(ts_Daily_Electricity[,1],
                                   end = length(ts_Daily_Electricity[,1])-(365-n_for))

# Create a subset for training purposes (from January 1, 2005 to December 31, 2009)
ts_Daily_Electricity_train <- subset(ts_Daily_Electricity_Feb,
                                     end = length(ts_Daily_Electricity_Feb)-n_for)

# Create a subset for testing purposes (from January 1, 2010 to February 28, 2010)
ts_Daily_Electricity_test <- subset(ts_Daily_Electricity_Feb,
                                   start = length(ts_Daily_Electricity_Feb)-n_for+1)


# Print first 5 and last 5 rows to cross-check with original Daily_Electricity df
#head(ts_Daily_Electricity_train, 5)
#tail(ts_Daily_Electricity_train, 5)

#head(ts_Daily_Electricity_test, 5)
#tail(ts_Daily_Electricity_test, 5)

# HOURLY
n_forH = 1416 # 59 days x 24 hours

# Create time series for data up until February 28, 2010
ts_Hourly_Demand_Feb <- subset(ts_Hourly_Demand_clean,
                                   end = length(ts_Hourly_Demand_clean)-(8760-n_forH))

# Create a subset for training purposes (from January 1, 2005 to December 31, 2009)
ts_Hourly_Demand_train <- subset(ts_Hourly_Demand_Feb,
                                   end = length(ts_Hourly_Demand_Feb)-n_forH)

# Create a subset for testing purposes (from January 1, 2010 to February 28, 2010)
ts_Hourly_Demand_test <- subset(ts_Hourly_Demand_Feb,
                                   start = length(ts_Hourly_Demand_Feb)-n_forH+24)
```

3 5 Best-Performing Models

3.1 Model 1: Neural Network with Hourly Data [p=24, P=0, K=c(1,1,1)]

```
# Fit NN Model
model1_fit <- nnetar(ts_Hourly_Demand_train, p=24, P=0,
                xreg=fourier(ts_Hourly_Demand_train, K=c(1,1,1)))

model1_forecast <- forecast(model1_fit, h=1416, xreg=fourier(ts_Hourly_Demand_train,
                                        K=c(1,1,1), h=1416))

# Fit and forecast with full data
model1_fit_full <- nnetar(ts_Hourly_Demand_clean, p=24, P=0,
                                xreg=fourier(ts_Hourly_Demand_clean, K=c(1,1,1)))

model1_forecast_full <- forecast(model1_fit_full, h=1416,
                                 xreg=fourier(ts_Hourly_Demand_clean,
                                         K=c(1,1,1), h=1416))

# Save forecast values
model1 <- as.data.frame(model1_forecast_full$mean) %>%
  rename(load_hourly = 1) %>%
  mutate(datetime = seq.POSIXt(from = as.POSIXct("2011-01-01 00:00:00"),
                               by = "hour",
                               length.out = n())) %>%
  mutate(date = as.Date(datetime)) %>%
  group_by(date) %>%
  summarise(load = round(mean(load_hourly)), .groups = "drop") %>%
  select(date, load)

# Remove last row (value for March 1, 2011)
model1 <- model1[-nrow(model1), ]
```

3.2 Model 2: Neural Network with Hourly Data [p=24, P=0, K=c(1,2,1)]

```
# Fit NN Model
model2_fit <- nnetar(ts_Hourly_Demand_train, p=24, P=0,
                xreg=fourier(ts_Hourly_Demand_train, K=c(1,2,1)))

model2_forecast <- forecast(model2_fit, h=1416, xreg=fourier(ts_Hourly_Demand_train,
                                        K=c(1,2,1), h=1416))

# Fit and forecast with full data
model2_fit_full <- nnetar(ts_Hourly_Demand_clean, p=24, P=0,
                                xreg=fourier(ts_Hourly_Demand_clean, K=c(1,2,1)))

model2_forecast_full <- forecast(model2_fit_full, h=1416,
                                 xreg=fourier(ts_Hourly_Demand_clean,
                                         K=c(1,2,1), h=1416))
```

```
# Save forecast values
model2 <- as.data.frame(model2_forecast_full$mean) %>%
  rename(load_hourly = 1) %>%
  mutate(datetime = seq.POSIXt(from = as.POSIXct("2011-01-01 00:00:00"),
                               by = "hour",
                               length.out = n())) %>%
  mutate(date = as.Date(datetime)) %>%
  group_by(date) %>%
  summarise(load = round(mean(load_hourly)), .groups = "drop") %>%
  select(date, load)

#remove last row (value for March 1, 2011)
model2 <- model2[-nrow(model2), ]
```

3.3 Model 3: Neural Network with Daily Data [p=3, P=0, K=c(2,2)]

```
# Fit NN Model
model3_fit <- nnetar(ts_Daily_Electricity_train, p=3, P=0,
                xreg=fourier(ts_Daily_Electricity_train, K=c(2,2)))

model3_forecast <- forecast(model3_fit, h=59, xreg=fourier(ts_Daily_Electricity_train,
                                        K=c(2,2), h=59))

# Fit and forecast with full data
model3_fit_full <- nnetar(ts_Daily_Electricity[,1], p=3, P=0,
                xreg=fourier(ts_Daily_Electricity[,1], K=c(2,2)))

model3_forecast_full <- forecast(model3_fit_full, h=59,
                                 xreg=fourier(ts_Daily_Electricity[,1],
                                          K=c(2,2), h=59))

# Save forecast values
model3 <- as.data.frame(model3_forecast_full$mean) %>%
  rename(load = 1) %>%
  mutate(date = seq.Date(from = as.Date("2011-01-01"),
                         by = "day",
                         length.out = 59),
         load = round(load)) %>%
  select(date, load)
```

3.4 Model 4: Neural Network with Daily Data [p=1, P=0, K=c(2,2)]

```
# Fit NN Model
model4_fit <- nnetar(ts_Daily_Electricity_train, p=1, P=0,
                xreg=fourier(ts_Daily_Electricity_train, K=c(2,2)))

model4_forecast <- forecast(model4_fit, h=59, xreg=fourier(ts_Daily_Electricity_train,
                                        K=c(2,2), h=59))
```

```
# Fit and forecast with full data
model4_fit_full <- nnetar(ts_Daily_Electricity[,1], p=1, P=0,
                 xreg=fourier(ts_Daily_Electricity[,1], K=c(2,2)))

model4_forecast_full <- forecast(model4_fit_full, h=59,
                                 xreg=fourier(ts_Daily_Electricity[,1],
                                              K=c(2,2), h=59))

# Save forecast values
model4 <- as.data.frame(model4_forecast_full$mean) %>%
  rename(load = 1) %>%
  mutate(date = seq.Date(from = as.Date("2011-01-01"),
                         by = "day",
                         length.out = 59),
         load = round(load)) %>%
  select(date, load)
```

3.5 Model 5: Neural Network with Daily Data [p=1, P=0, K=c(1,2)]

```
# Fit NN Model
model5_fit <- nnetar(ts_Daily_Electricity_train, p=1, P=0,
                 xreg=fourier(ts_Daily_Electricity_train, K=c(1,2)))

model5_forecast <- forecast(model5_fit, h=59, xreg=fourier(ts_Daily_Electricity_train,
                                          K=c(1,2), h=59))

# Fit and forecast with full data
model5_fit_full <- nnetar(ts_Daily_Electricity[,1], p=1, P=0,
                 xreg=fourier(ts_Daily_Electricity[,1], K=c(1,2)))

model5_forecast_full <- forecast(model5_fit_full, h=59,
                                 xreg=fourier(ts_Daily_Electricity[,1],
                                              K=c(1,2), h=59))

# Save forecast values
model5 <- as.data.frame(model5_forecast_full$mean) %>%
  rename(load = 1) %>%
  mutate(date = seq.Date(from = as.Date("2011-01-01"),
                         by = "day",
                         length.out = 59),
         load = round(load)) %>%
  select(date, load)
```

4 Accuracy Results (vs Validation Set)

```
# Model 1
model1_scores <- accuracy(model1_forecast$mean, ts_Hourly_Demand_test)

# Model 2
```

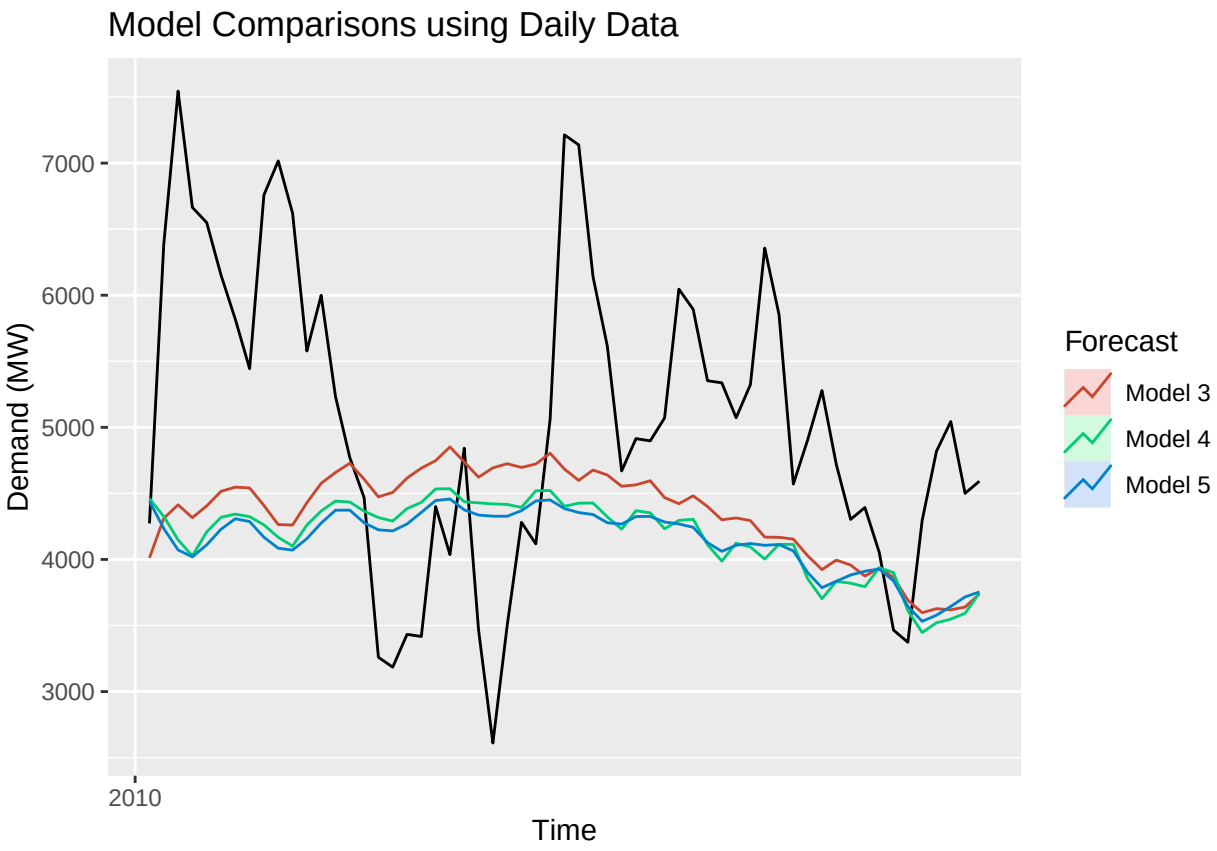
```r
model2_scores <- accuracy(model2_forecast$mean, ts_Hourly_Demand_test)

# Model 3
model3_scores <- accuracy(model3_forecast$mean, ts_Daily_Electricity_test)

# Model 4
model4_scores <- accuracy(model4_forecast$mean, ts_Daily_Electricity_test)

# Model 5
model5_scores <- accuracy(model5_forecast$mean, ts_Daily_Electricity_test)

# Create data frame
scores <- as.data.frame(
  rbind(model1_scores, model2_scores, model3_scores, model4_scores, model5_scores)
  )
row.names(scores) <- c("Model 1", "Model 2","Model 3","Model 4", "Model 5")
```

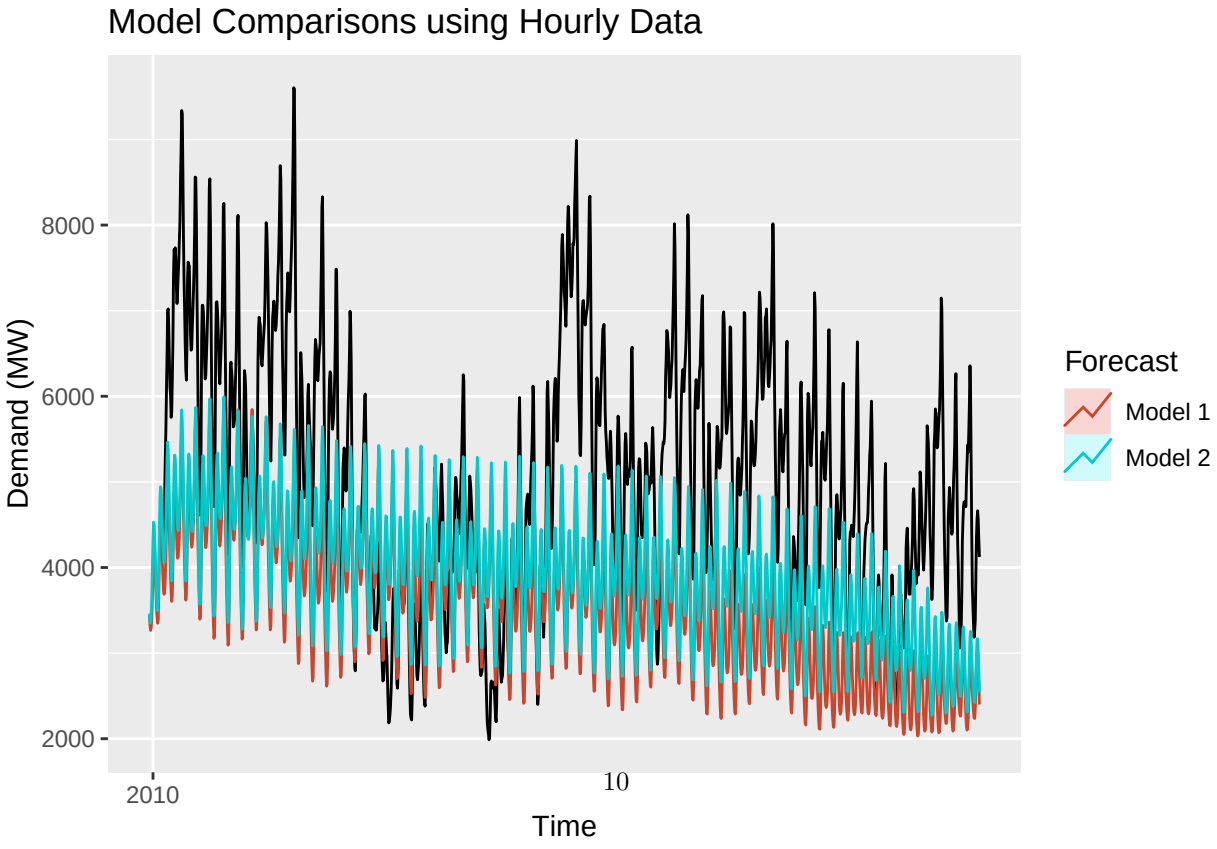
Table 1: Forecast Accuracy for Load Using Validation Data Set

	ME	RMSE	MAE	MPE	MAPE	ACF1	Theil's U
Model 1	1544.5355	1976.333	1694.715	26.39703	31.58121	0.98458	4.56019
Model 2	1144.6851	1665.890	1393.783	17.83923	26.12010	0.98487	3.87630
Model 3	698.8587	1355.250	1122.563	9.04491	21.76706	0.78798	1.62147
Model 4	876.0507	1449.754	1209.301	12.71530	22.87845	0.76365	1.66323
Model 5	903.2318	1458.905	1204.185	13.31564	22.55376	0.76387	1.64984

5 Model Comparisons (Plots)

5.1 Training and Validation Sets





5.2 2011 Forecasts with Full Data Set